Язык запросов STEP_QL

Top Pr

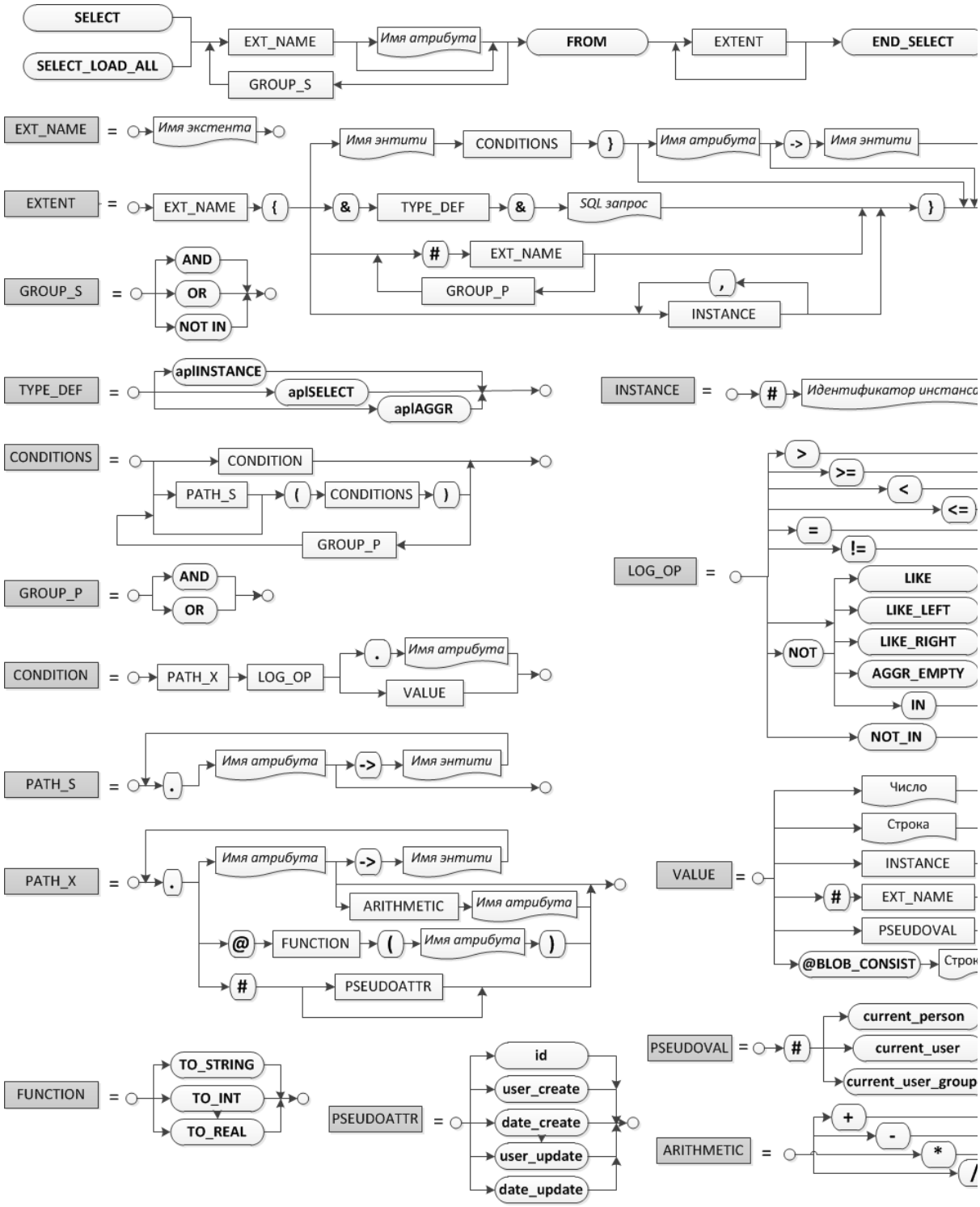
Назначение
1.Спецификация
2.Примеры запросов
3.Выполнение запроса

Назначение

Язык запросов STEP_QL_d предназначен для проведения сложного многопланового поиска объектов в базе данных. Язык позволяет запрос на уровне типов данных (entity) и их атрибутов, задать условия на конкретные атрибуты entity. Возможно определения условия на с пути атрибутов и на комбинации путей. Возможные имена entity и атрибутов, а так же набор атрибутов каждой entity и типы значений ат определяются действующей EXPRESS-схемой базы.

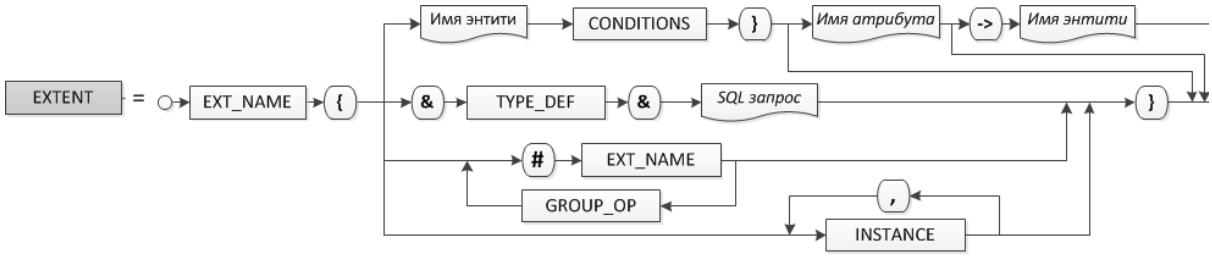
Спецификация

Запрос состоит из неограниченного (в пределах разумного :-) количества подзапросов, объединенных групповыми условиями. Графическая синтаксическая схема запроса приведена на следующем рисунке.



Запрос разбит на две части: между ключевыми словами FROM … END_SELECT описываются подзапросы, а между ключевыми SELECT ... FROM (или SELECT_LOAD_ALL ... FROM) формируется выходной экстент путем объединения и (или) пересечения результатов подзапросов.

Если запрос начинается с ключевого слова SELECT, то на клиента загружается только список инстансов. Если запрос начинает конструкцией SELECT_LOAD_ALL, то параллельно с выполнением запроса в клиентский кеш загружаются все инстансы, полученные в резуль подзапросов (а не только указанные в первой секции) вместе со всеми атрибутами.



Подзапрос может быть задан несколькими способами:

1) Подзапрос на основе ентити и условий на атрибуты. Каждый такой подзапрос, по сути, описывает один «куст» условий, начинающи определенной корневой entity, например, от approval-а или apl_characteristic_value (см примеры). Каждая «ветка» «куста» представляет собой из атрибутов и заканчивается условием на конкретный атрибут; см примеры 1 - 3. Вместо атрибута в условии можно использовать псевдоатри #user_create, #user_update, #date_create, #date_update (см таблицу с описанием псевдоатрибутов и пример 5). Если цепочка условий на атриб задана - то результатом подзапроса будут все instance указанной entity. Если в конце цепочки нет условия, то проверяется соответствие типу последней указанной entity. Цепочки атрибутов могут в любом порядке ветвиться с использованием логических операций AND и OR, ветви до разделяться круглыми скобками.

При задании условий можно сравнивать атрибуты инстанса одной энтити (атрибуты должны быть одного типа), подробнее см пример можно использовать арифметические выражения (например, можно вычесть друг из друга атрибуты, в которых хранятся стоимость предполаг закупочная, и сравнивать с некоторым числом). Использование второго операнда-атрибута и арифметических операций недопустимо для атр типа aplAGGR (см таблицу допустимых отношений).

Строковые значения могут заключаться как в одинарные, так и в двойные кавычки. При сравнении строк по умолчанию регистр симво значения не имеет.

Если тип атрибута - строка, а сравнивать их значения надо как числа (или наоборот), то можно перед сравнением преобразовать знач атрибута к другому типу с помощью функций @TO_REAL, @TO_INT и @TO_STRING. Функция пишется после точки, в скобках - имя преобра атрибута. Подробнее см пример 7.

При использовании в качестве значения (“VALUE”) подзапроса (#EXT_NAME) допустимы только операции «входит» (“IN”) и «не входит» (“NOT_IN”) - и только для атрибутов типа «инстанс» и «select из инстансов», а так же для идентификаторов инстансов (псевдоатрибут таблицу допустимых отношений). Используемый подзапрос должен быть определен до использования.

Если надо сравнить атрибуты типа aplINSTANCE с пустым или не заданным значением, то для сравнения надо использовать значение

При сравнении атрибутов типа aplINSTANCE можно использовать псевдозначения .

В качестве значения так же может использоваться строка (строки), содержащаяся (-иеся) в БЛОБах (пока конструкция работает толь сервера ORACLE; при этом надо в «Администраторе БД для ORACLE» создать индексы для БЛОБов). Для этого используется конструкция @BLOB_CONSIST (“строковое значение”) или @BLOB_CONSIST (“строковое значение 1”,“строковое значение 2” …); допустимое отношение - р (‘=’). При обработке такого условия сервер определит идентификаторы всех БЛОБов, в которых встречаются заданные строковые значения (р символов не имеет значения); переведет идентификаторы в строчный вид и сравнит с каждым идентификатором значения заданных атрибутс

Результатом подзапроса является экстент инстансов типа корневой ентити, атрибуты или псевдоатрибуты или цепочки атрибутов кот удовлетворяют заданным условиям. Если после описания такого экстента указан атрибут корневого ентити подзапроса, то в выходной экстент попадает не инстансы корневого типа, а инстансы на которые они ссылаются указанным атрибутом. Если же после атрибута указано имя уто энтити, то в выходной список попадают только инстансы такого типа.

Таблица псевдоатрибутов (PSEUDOATTRS)

Имя	Тип	Тип сервера	Описание
#id, #	aplINSTANCE	Lite, Oracle	идентификатор (внутренний номер) инстанса
#user_create	aplINSTANCE	Oracle	пользователь, создавший инстанс (идентификатор инстанса типа apl соответствующий пользователю)
#user_update	aplINSTANCE	Oracle	послений пользователь, изменивший какой-либо атрибут ин (идентификатор инстанса типа apl_user, соответствующий пользователю)
#date_create	aplSTRING	Oracle	дата создания инстанса
#date_update	aplSTRING	Oracle	дата последнего изменения атрибута инстанса

Таблица псевдозначений (PSEUDOVAL)

Имя	Описание
#current_person	В запросе будет заменено на идентификатор инстанса текущего сотрудника

Таблица допустимых отношений

Тип определяемого атрибута		Допустимое значение (Пример)	Допустимые отношения
Число (aplINTEGER, aplREAL) Строка (aplSTRING), преобразованная к числу (@TO_INT) или (@TO_REAL)		Число (345,10.202)	‘<’,’<=’,’>’,’>=’,’=’,’!=’
Строка (aplSTRING) Число, преобразованное к строке (@TO_STRING)		Строка (‘Подшипник‘)	‘<’,’<=’,’>’,’>=’, ‘=’,’ =’,’IN’,’LIKE’, ’LIKE_LEFT’,’LIKE_RIGH
		Содержимое БЛОБов	‘=BLOB_CONSIST (xxx)’
Булевое (aplBOOL)		Строка: TRUE или FALSE	‘=’,’!=’
Инстанс (aplINSTANCE,aplSELECT)		Значение instance (#34095 или #0 для поиска не определенных значений) или псевдозначение(например #current_user для поиска текущего пользователя)	‘=’,’!=’
		Значение подзапроса (#Ext_0)	‘IN’,’NOT_IN’
AGGR (aplAGGR) из	инстанс	Значение instance (#34095)	‘LIKE’
	число	Число (345,10.202)	
	строка	Строка (‘Подшипник‘)	
	любой тип	**значение не требуется**	'AGGR_EMPTY', 'NOT AGGR_EMPTY'
Псевдоатрибуты #, #id, #user_create, #user_update		Значение подзапроса (#Ext_0)	‘IN’,’NOT_IN’
		Значение instance (#34095 или #0 для поиска не определенных значений) или псевдозначение(например #current_user для поиска текущего пользователя)	‘<’,’<=’,’>’,’>=’, ‘=’,’!=’
Псевдоатрибуты #date_create, #date_update		Строка с датой в формате ГГГГММДДччммсс (например, 12 часов 20 минут 1 февраля 1998 года: '19980201122000')	‘<’,’<=’,’>’,’>=’, ‘=’,’!=’

2) Подзапрос, заданный списком инстансов. Между фигурными скобками через запятую указываются идентификаторы инстансов с пр - символом решетка. Результатом подзапроса является экстент указанных инстансов. Такие подзапросы, как правило, определяют список инс для использования в последующих подзапросах. Например `Ext1{#3902, #3901}`

3) Подзапрос, заданный объединением и/или пересечением ранее определенных подзапросов. Например `Ext2{#Ext0 OR #Ext1}`

4) Подзапрос, заданный SQL-запросом (допустим только для сервера приложений для oracle). Запрос на языке SQL должен быть запр «SELECT»; запрос должен возвращать выборку значений из одного (только одного!) столбца атрибута типа aplINSTANCE или aplSELECT или aplAGGR. Описание типа может быть "aplINSTANCE", "aplSELECT" или "aplAGGR" и должно соответствовать типу выбираемого атрибута. Подр см пример 8.

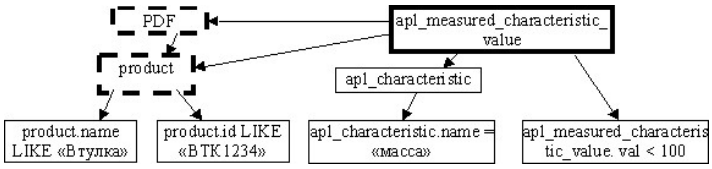
Общий результат запроса получается комбинированием множеств каждого экстента, полученных последовательным взятием атрибут описанных цепочкой атрибутов, от каждого инстанса экстента подзапроса. При комбинировании можно использовать групповые операций ANI (пересечение результатов экстентов), OR (объединение результатов экстентов) и NOT IN (вычитание результатов экстентов). Выходная комби описывается между ключевыми словами SELECT … FROM или SELECT_LOAD_ALL … FROM. Если результат представляет собой сложную ком множеств, рекомендуется использовать скобки для корректной группировки операций.

Разделителями ключевых слов и операндов являются любые комбинации следующих символов: пробел, перевод строки; символ табу Для ключевых слов регистр символов не учитывается, т.е. можно написать «SELECT … FROM» или «select … from» и т.п. В текст запроса можно вставлять комментарии, аналогичные используемым в языке C (после символов ‘//’ комментарий до конца строки; от символов ‘/*’ комментари символов ‘*/’).

Для оптимизации скорости выполнения запросов рекомендуем объединять условия на атрибуты инстансов одного типа в отдельную в подробнее см. пример 1.

Примеры запросов

Пример 1



где пунктиром показаны типы объектов, входящих в результат запроса, толстой линией показан исходный тип подзапроса Ext_1.
Запрос на языке запросов будет иметь вид:

```
SELECT Ext_1.item FROM
Ext_1{ apl_measured_characteristic_value
(
    .val < 100 AND
    .characteristic->apl_characteristic.name = 'МАССА' AND
    (
        .item->product_definition_formation.of_product->product
        (
            .name LIKE 'ВТУЛКА' AND
            .id = 'BTK1234'
        ) OR
        .item->product
        (
            .name LIKE 'ВТУЛКА' AND
            .id = 'BTK1234'
        )
    )
)
}
END_SELECT
```

Следует иметь в виду, что в такой запрос вернет экстент из instance как типа product, так и типа product_definition_formation. Как видно из прим
скобки используются не только для определения порядка выполнения логических операций, но и для группировки условий на атрибуты одной
Например, определение подзапроса

```
Ext_1{ apl_measured_characteristic_value
.item->product_definition_formation.of_product->product
(
    .name LIKE 'ВТУЛКА' AND
    .id = 'BTK1234'
)
}
```

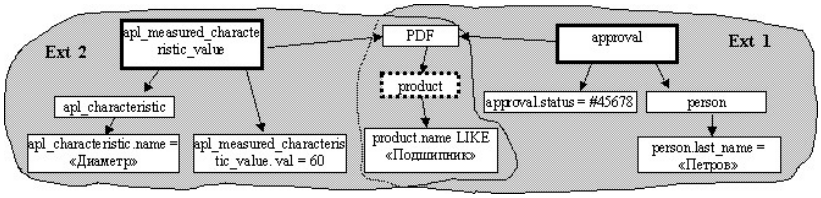
по смыслу эквивалентно следующему определению:

```
Ext_1{ apl_measured_characteristic_value
    .item->product_definition_formation.of_product->product
    .name LIKE 'ВТУЛКА' AND
    .item->product_definition_formation.of_product->product
    .id = 'BTK1234'
}
```

Однако первая запись компактнее, а кроме того второй вариант будет выполняться дольше за счет двукратного прохождения по цепочке
apl_measured_characteristic_value ->product_definition_formation ->product . Если есть возможность, надо группировать условия на атрибуты одн

Пример 2.

Более сложная задачка: найти все изделия (только изделия!), название которых начинается с «Подшипник», у которых есть версии со статусо
присвоенным конструктором Петровым и диаметром наружного кольца 60 мм. При этом instance типа статуса (approval_status) известен; его id
45678.
График entity участвующих в запросе имеет два «куста», начинающихся с apl_measured_characteristic_value и с approval, и имеющих общую ве
product-pdf :



пунктиром показаны типы объектов, входящих в результат запроса, толстой линией показан исходные типы для подзапросов Ext_1 и Ext_2.

```
SELECT Ext_1.of_product AND Ext_2.of_product FROM
Ext_1{ apl_approval_person_organization
(
    .status = #45678 AND
    .person_organization->person.last_name = 'Петров' AND
    .item->product_definition_formation.of_product->product.name LIKE 'ПОДШИПНИК'
)
}.item->product_definition_formation
Ext_2{ apl_measured_characteristic_value
(
    .val = 60 AND
    .characteristic->apl_characteristic.name = 'Диаметр' AND
    .item->product_definition_formation.of_product->product.name LIKE 'ПОДШИПНИК'
)
}.item->product_definition_formation
END_SELECT
```

Этот запрос возвращает экстент из инстансов типа product.
Вообще подзапрос типа

```
Ext_1{ apl_approval_person_organization
    .item->product_definition_formation.of_product->product.name LIKE 'ПОДШИПНИК'}
```

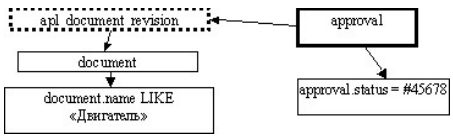
вернет только те статусы, которые указывают на версии изделий. Если надо найти все статусы (и т.п.), принадлежащие именно изделиям (а не версиям) и параметры изделий неизвестны, то в определении подзапроса должна присутствовать следующая ветвь:

```
.item->product
```

Результат запроса образуется пересечением результатов двух подзапросов; в результате запрос вернет изделия, имеющие одновременно и с характеристику. Таким образом можно выполнять поиск на любом количестве «кустов». «Верхушкой куста» могут быть, например, entity apl_characteristic_value (для характеристик), apl_document_reference (для документов), approval (для утверждений) и т.п. Возможен так же поис например, по одному уровню сборки - тогда вершиной «куста» будет product_definition_relationship.

Пример 3.

Найти документы со словом «Двигатель» в названии, которые не имеют статуса «Утверждено» ни для одной своей версии (статусы, присвоен собственно объекту «документ» для простоты не рассматриваем). Id статуса равен 4058.
Для такого поиска необходимо использовать исключающие условия поиска. Сначала определяется подзапрос Ext_0, условия которого начина объекта «статус», а результат представляет собой версии документа, имеющие требуемый статус:



где пунктиром показаны типы объектов, входящих в результат экстента Ext_0, толстой линией показан исходные типы для запросов Ext_0.
Далее определяется вспомогательный подзапрос Ext_0_a, определяющий документы, имеющие версии, входящие в предыдущий экстент.
Последним определяется подзапрос Ext_0_b. состоящий из документов, не входящих в результат подзапроса Ext_0_a, и из которого формиру результаты запроса:

```
SELECT
Ext_0_b
```

```
}.item->apl_document_revision

Ext_0_a {apl_document_revision
    .# IN #Ext_0
}.of_document->apl_document

Ext_0_b {apl_document
(
    .name LIKE 'Двигатель' AND
    .# NOT_IN #Ext_0_a
)
}
END_SELECT
```

Пример 4.

Найти все версии изделий (product_definition_formation), у которых есть статусы (apl_approval_person_organization).
Графически условие на подзапрос выгладит так:

Запрос на языке запросов будет иметь вид:

```
SELECT Ext_1 FROM
    Ext_1{ apl_approval_person_organization.item ->product_definition_formation}.item
END_SELECT
```

Пример 5.

Найти все версии изделий (product_definition_formation), атрибуты которых последним менял пользователь с id 3902 (главный администратор E
1 мая 2016 года. Такой запрос возможен только на сервере приложений для oracle:

```
SELECT Ext_1 FROM
    Ext_1{product_definition_formation(.#user_update = #3902 AND .#date_update > '20160501000000')}
END_SELECT
```

Так же можно использовать подзапрос с SQL:

```
SELECT Ext_1 FROM
    Ext_1{&aplINSTANCE& select INSTANCE_ID from e000000 where USER_ID=3902 AND DATE_OF_UPDATE > '20160501000000
END_SELECT
```

Пример 6.

Найти все сообщения (apl_message) у которых дата отправки меньше даты получения:

```
SELECT
    Ext
FROM
    Ext {apl_message.date_send_s < .date_received_s}
END_SELECT
```

Пример 7.

Найти все версии изделий (product_definition_formation), у изделий (product) которых атрибут code больше 10. Сравнивать надо как числа, а не строки.

```
SELECT Ext_1 FROM
    Ext_1{product.@TO_INT(code) > 20}
END_SELECT
```

Пример 8.

Найти сотрудника для пользователя с идентификатором 13089 и все объекты, входящие в папку с идентификатором 48160 с помощью SQL.

```
SELECT ExtSA AND ExtSE FROM
    ExtSE{&aplINSTANCE&SELECT A000433 from apl_user.e000059 where instance_id=13089}
    ExtSA{&aplAGGR&SELECT A000428 from apl_user.e000058 where instance_id=48160}
```

```
SELECT ExtF FROM
     ExtF{apl_folder.content aggr_empty}
END_SELECT
```

Пример 10.

Найти все документы, взятые на редактирование текущим сотрудником

```
SELECT
      Ext
FROM
      Ext {apl_digital_document.author = #current_person}.of_document
END_SELECT
```

Выполнение запроса

Выполнение запросов можно осуществлять с помощью утилиты AplDBE.exe (там же можно посмотреть списки существующих entity и их instan
PSM (в режиме расширенного поиска), либо программно.

Выполнение запроса в AplDBE

Утилита AplDBE позволяет запускать на выполнение описанные выше запросы. Необходимо после присоединения к БД выполнить команду «
>Find» (или Ctrl+F).
Появится окно, в котором можно набрать текст запроса, запустить его на выполнение, сохранить или загрузить из текстового файла текст зап
Список найденных instance выводится в разделе “Query list”. Перед выполнением запроса программа проверит правильность синтаксиса текст
запроса и укажет на допущенные ошибки. Таким образом, эта возможность может быть использована, например, для отладки текста запроса
его программной реализацией.

Выполнение запроса в PSM

Расширенный поиск («Поиск в БД») в PSM также реализован с помощью описываемого языка запросов. В PSM имеется возможность просмот
редактирования текста запроса расширенного поиска, который формируется с помощью визуальных форм. Для этого необходимо:
1.В меню «Настройки->Отображение->Двойной клик на элементе дерева» установить переключатель в поле «Открывает свойства объекта»
2.Сформировать расширенный запрос с помощью команды «Поиск в БД». Результат выполнения команды отображается в разделе «Расшире
запросы» основного окна модуля PSM как «расширенный запрос № N»
3.Удерживая обе клавиши «ALT» щелкнуть дважды на сформированном расширенном запросе
Появится окно редактирования, такое же, как в AplDBE, с сформированным программой текстом запроса. Эту возможность удобно использов
например, для построения относительно простого шаблона запроса (для его последующего усложнения с помощью языка запросов), или для
самого языка запросов.
Текст запроса может быть изменен, и запрос будет выполняться уже с новыми условиями. Но необходимо помнить, что PSM умеет отображат
объекты тех типов, которые перечислены в форме «Поиск в БД» - product, product_definition_formation, apl_business_process, и т.п., то есть те,
задано соответствующее визуальное представление (иконка). Объекты же, которые осуществляют связь этих объектов (типа
product_definition_relationship) могут быть найдены, но не отображены.

Программное выполнение запроса

Для выполнения запроса предназначены две функции класса CaplNetStepData - `NET_QueryEditParse` и `NET_QueryExecute`.

```
bool NET_QueryEditParse(CString *query_lng,bool is_dlg=false,CString *errmsg=0);
```

Функция осуществляет редактирование, разбор и перевод во внутренний формат выражения, описанного на языке запросов STEP_QL. Парам

query_lng	(IN)	принимает текст запроса;
is_dlg	(IN)	если параметр равен true, то отображается диалог редактирования запроса, в который заносится текст запроса из query_lng; при false диалог не отображается; значение по умолчанию false.
errmsg	(OUT)	если параметр указывает на строку, то в случае ошибки в него заносится текстовое описание ошибки; значение по умолчанию 0.

При удачном выполнении функция возвращает true; в случае ошибки функция возвращает значение false. Описание ошибки можно получить с
помощью параметра errmsg, а так же с помощью функции GetLastErrorDescription

ext	(OUT)	экстент, содержащий результаты запроса.

При удачном выполнении функция возвращает true; в случае ошибки функция возвращает значение false. Описание ошибки можно получить с помощью функции GetLastErrorDescription